POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

SayCan For Pick and Place Operation

Machine Learning

Authors:
Andrea Galenda (10707919)
Daniele Firmani (10726969)
Dario Pacini (10769966)

Professor: Prof. Loris Roveda

Academic Year: 2024-25

Contents

Contents **i**

Introduction **1**

1 Initial environment **2**
1.1 Pick and place targets 2
1.2 ViLD 2
1.3 Decision-making logic 3
1.3.1 LLM scoring 3
1.3.2 Affordance scoring 4
1.3.3 Combined scoring 4

2 Environment enhancements **5**
2.1 Environment Enhancement 5
2.1.1 Introduction of New Objects 5
2.1.2 Object Classification with ViLD 6
2.1.3 Dataset Generation for Manipulation 6
2.1.4 Training CLIPort on New Object Set 6
2.1.5 Evaluation: Pick-and-Place Tasks 7

3 Feedback **8**
3.1 ViLD improvement 8
3.2 Dynamic scene update and feedback 8
3.3 Fixing the "Stack all the blocks" task 9
3.3.1 Previous Limitation: Lack of Task Completion Awareness 9
3.3.2 Task Completion Verification: Stack All the Blocks 10

Introduction

The project builds upon a pre-existing environment consisting of colored objects (blocks and bowls) and a robotic arm already programmed to interact with them. It has been extended to allow the arm to interact with new types of objects, include an iterative visual feedback system, and use a more robust scoring mechanism. All improvements were integrated by modifying existing functions, updating their interaction within the code, and creating new objects for the robotic arm to recognize.

1 Initial environment

1.1. Pick and place targets

The objects that the system was initially able to recognize were two: blocks (to pick up) and bowls (to place onto). The targets can have different colors, but a fixed shape — cubic for the blocks and one consistent shape for the bowls.

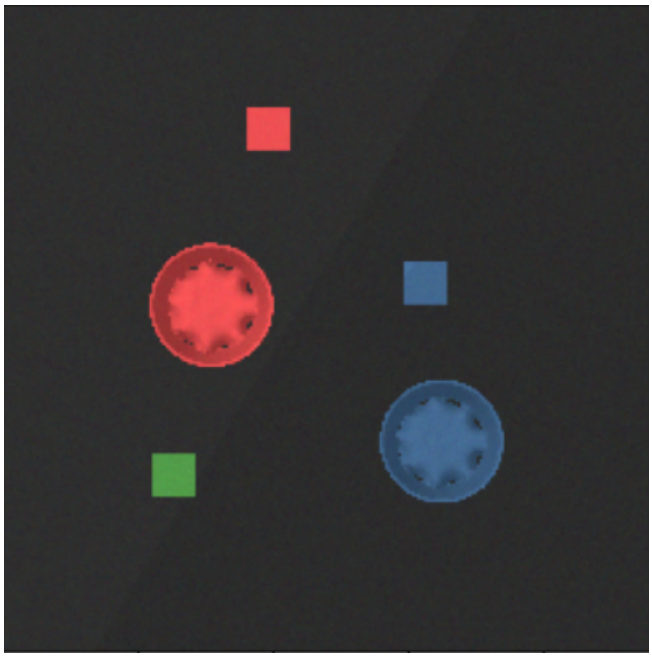


Figure 1.1: An example of an environment with three blocks and two bowls.

1.2. ViLD

ViLD is a function that processes RGB images captured by the system's top-down cameras in order to detect and classify the objects present in the scene. It provides semantic labels and bounding boxes for each recognized item, which are then used by the scoring mechanism to evaluate the best next action to take based on the user's instruction.

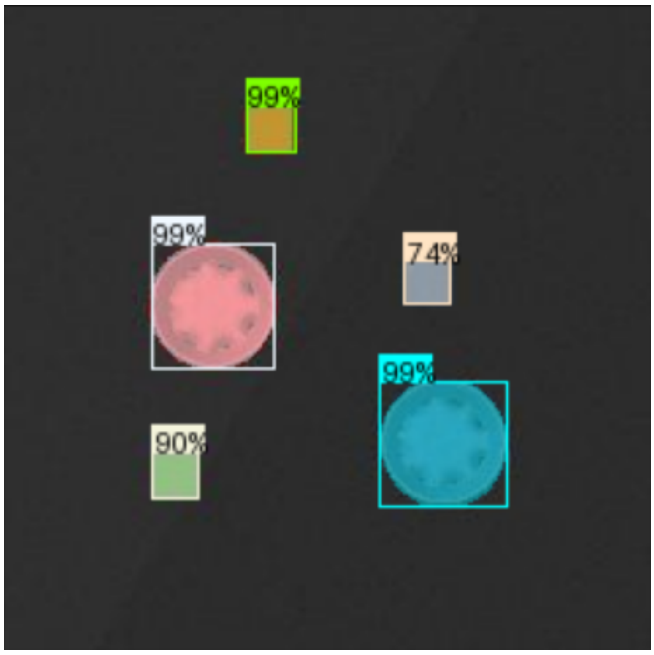


Figure 1.2: Image processed by ViLD.

1.3. Decision-making logic

The decision-making process behind the robot arm's actions is guided by a scoring system that evaluates the available action options at each step. This system is based on a combination of two distinct yet complementary approaches: a language-based score generated by a Large Language Model (LLM), and a perception-based score derived from the actual visual feasibility of the action (affordance). By combining these two scores, the system selects the most appropriate action in a given context.

1.3.1. LLM scoring

The language model is prompted with: the user input (the task to fulfill), the list of available action options (e.g., "pick the red block and place it on the green bowl"), a text-based scene description derived from the ViLD perception system.
For each option, the LLM estimates the likelihood that it represents the correct next step, given the request and the current context. The result is a normalized score that reflects the semantic coherence between the proposed action and the user's intent.

1.3.2. Affordance scoring

In parallel, an affordance score is computed based on the presence of the required objects in the observed scene. This score evaluates whether the objects involved in a given action are actually detected and visible in the ViLD-processed image. If an action references objects not present in the current scene, it receives a zero score; if it is in the scene, an un itary score.

1.3.3. Combined scoring

Finally, the two scores are combined through multiplication.
This approach penalizes actions that may be linguistically appropriate but physically impossible (e.g., "pick up the blue block" when the blue block is not visible), and favors those that are both aligned with the instruction and feasible in the environment.
The action with the highest combined score is then selected as the next step to execute.

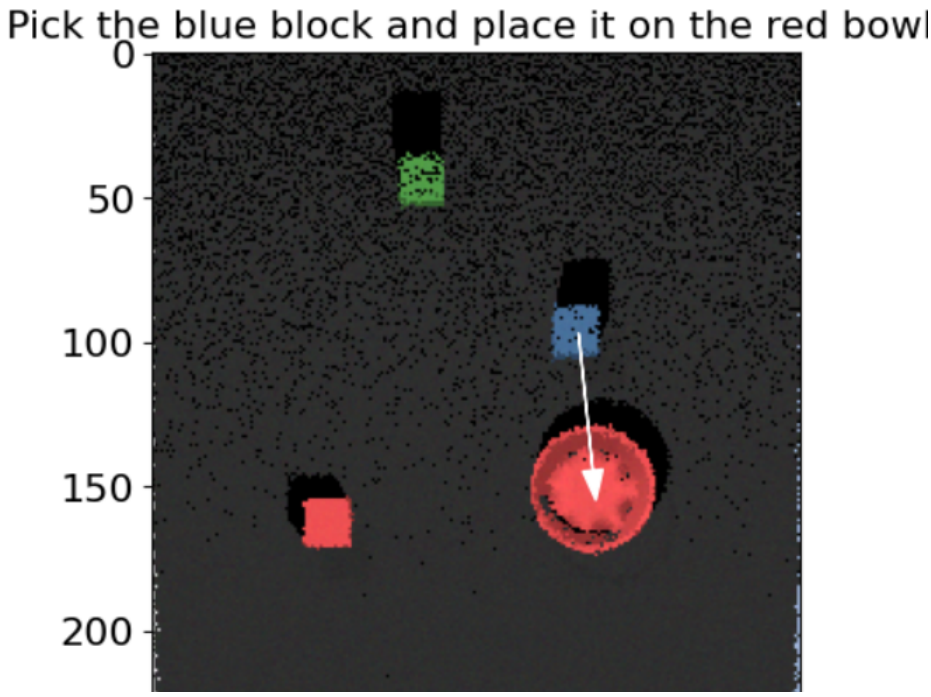


Figure 1.3: Action selection.

2 | Environment enhancements

2.1. Environment Enhancement

This chapter presents the procedure and results of enriching the robot's environment to improve the system's capability to handle novel manipulation tasks. The process is structured into several stages, involving visual grounding, dataset generation, and retraining of the manipulation policy.

2.1.1. Introduction of New Objects

To evaluate the scalability and adaptability of the manipulation pipeline, new objects were introduced into the environment. These items were not present during the original training of the manipulation policy and were selected to represent different shapes, textures, and colors, in order to test generalization capabilities.

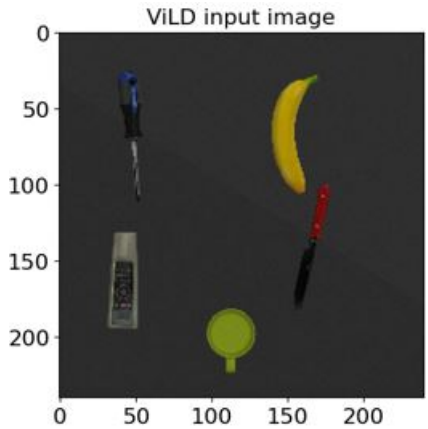


Figure 2.1: Overview image of new objects in the simulated or real environment

2.1.2. Object Classification with ViLD

The newly introduced items were identified and classified using the ViLD (Vision-Language Detection) model. This model enables open-vocabulary object detection based on language prompts, providing flexibility for detecting previously unseen categories without retraining.

The output of ViLD provides bounding boxes and labels grounded in natural language, which are used to extract the spatial locations of relevant objects in the scene.

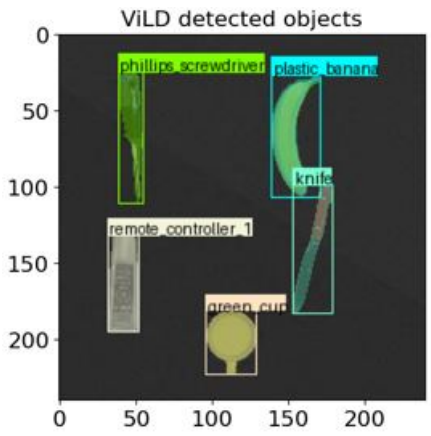


Figure 2.2: ViLD bounding box results on scene with new objects

2.1.3. Dataset Generation for Manipulation

Based on the ViLD detections, a small dataset was generated to enable the system to learn how to manipulate the newly introduced items. This dataset includes RGB images, language descriptions, object positions, and end-effector poses for each grasp attempt.

The dataset was curated to be consistent with the format used by CLIPort, ensuring compatibility with the existing vision-language manipulation framework.

2.1.4. Training CLIPort on New Object Set

CLIPort, the end-to-end vision-language policy used for manipulation, was fine-tuned using the new dataset. This training phase aimed to teach the model how to interpret ViLD-based instructions and execute grasp-and-place actions on the newly introduced objects.

2.1.5. Evaluation: Pick-and-Place Tasks

To validate the effectiveness of the updated system, we tested the trained CLIPort policy in a series of pick-and-place tasks involving the newly classified objects. The system was instructed via natural language commands referencing the ViLD-detected categories.

The evaluation measured success rate, spatial precision, and task completion time. The results confirmed that the combined use of ViLD and CLIPort allows for accurate manipulation of previously unseen objects with limited training data.

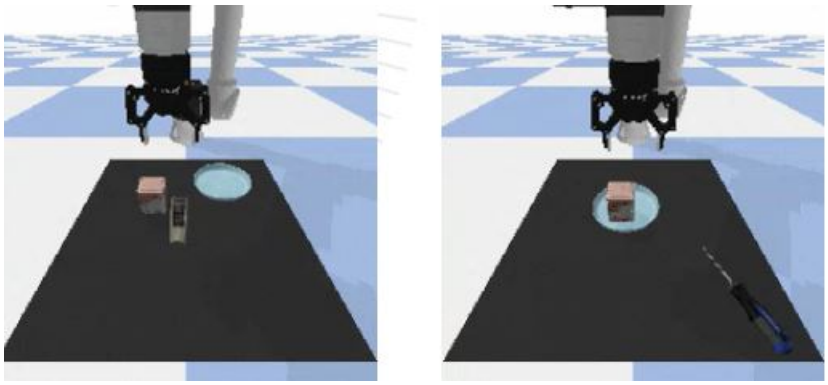
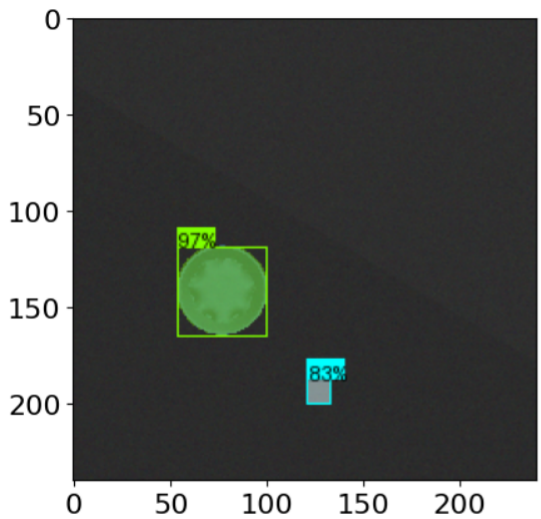


Figure 2.3: Before-and-after image of a successful pick-and-place operation

3 Feedback

3.1. ViLD improvement

To implement an iterative feedback system, the `vild` function was directly modified. In particular, it was adjusted to also output the positions of the objects detected in the scene.




```
Found Objects and positions from Vild:  blue bowl → is positioned in the environment at these coordinates (x,y): (0.082, -0.407)
red block → is positioned in the environment at these coordinates (x,y): (0.222, -0.542)
```

Figure 3.1: Added position output in the `vild` function.

3.2. Dynamic scene update and feedback

At each iteration, a new image is captured and processed using ViLD. The detected objects and their positions are embedded into an updated scene description, which is passed to the language model as part of the prompt. This ensures that the LLM's decisions are grounded in the current state of the environment. Let's an application for the task "Put any blocks on their matched colored bowls".

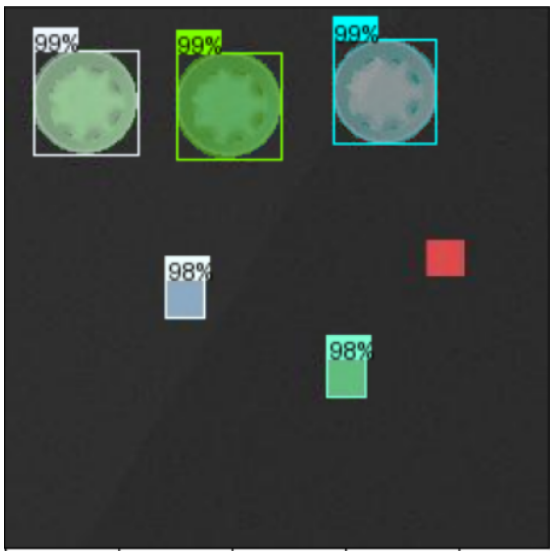


Figure 3.2: First step.

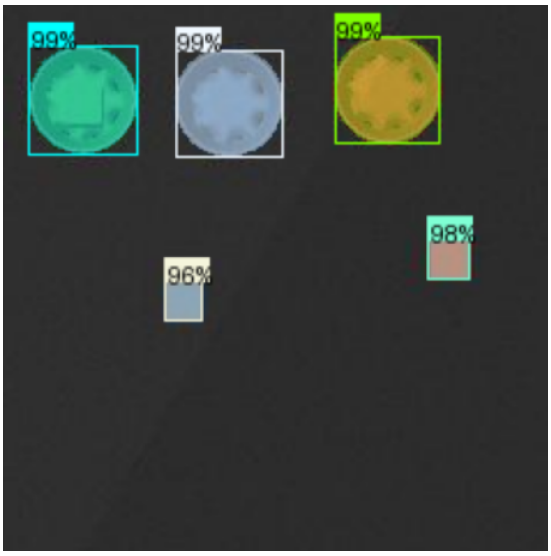


Figure 3.3: Second step.

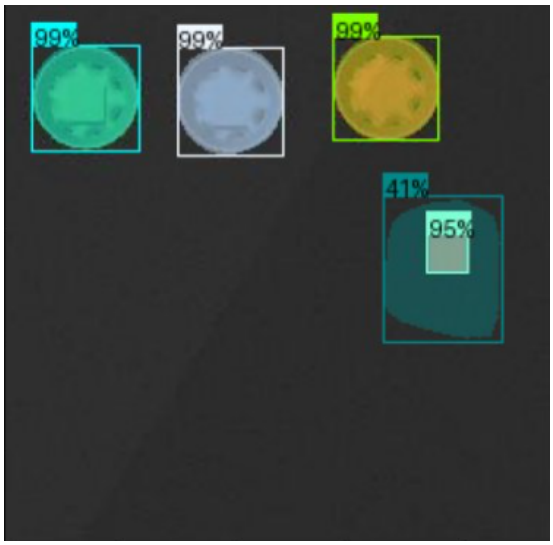


Figure 3.4: Third step.

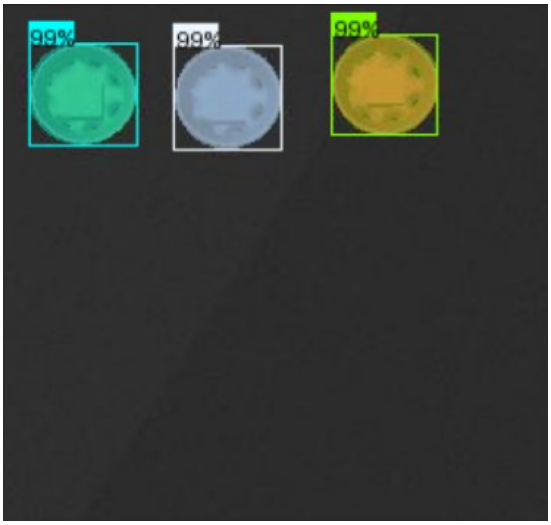


Figure 3.5: Fourth step.

3.3. Fixing the "Stack all the blocks" task

3.3.1. Previous Limitation: Lack of Task Completion Awareness

Before the integration of a dedicated feedback verification function, the system was unable to autonomously recognize the successful completion of the "Stack all the blocks" task. Although the robot was able to stack the blocks correctly through successive actions, it lacked a mechanism to evaluate the final configuration.

As a result, the LLM-based planner continued to suggest further actions even when the goal was already achieved. This often led to unnecessary or incorrect movements, such as attempting to re-pick a block or moving one that was already in place—ultimately causing the previously stacked blocks to fall and the task to fail.

This limitation highlighted the need for a more robust and geometric condition to define task success and terminate the action sequence at the appropriate moment.

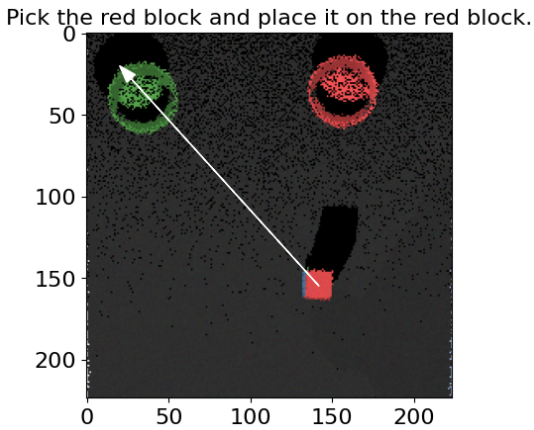


Figure 3.6: Wrong action choice.

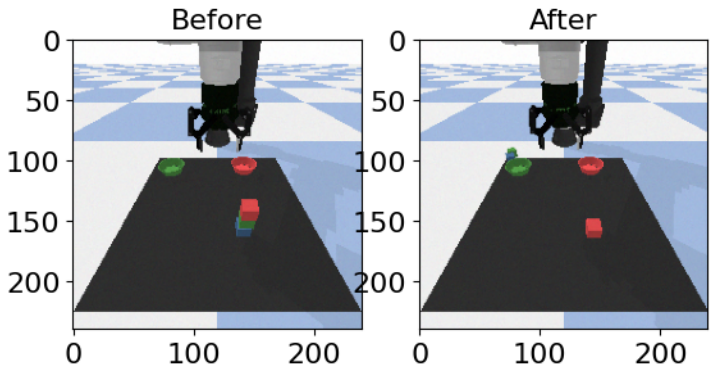


Figure 3.7: Task failed.

3.3.2. Task Completion Verification: Stack All the Blocks

The task "Stack all the blocks" was successfully implemented by integrating a custom feedback verification function that dynamically evaluates the relative positions of the blocks after each action. Specifically, a function named `areblocksstacked` was introduced to assess whether all blocks are spatially aligned within a specified threshold in the xy-plane.

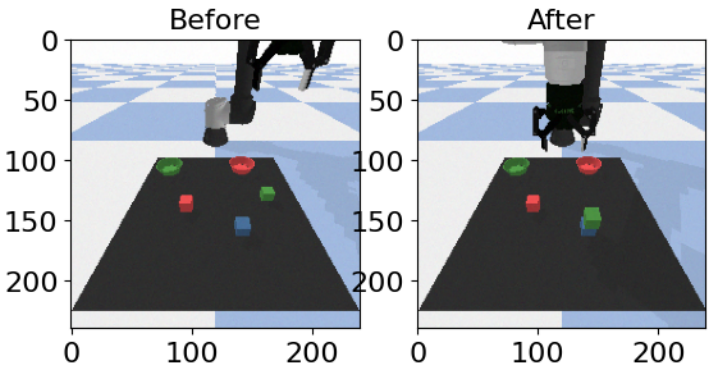


Figure 3.8: First action.

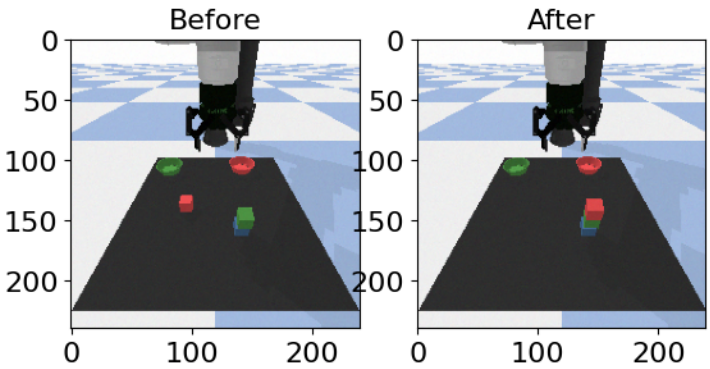


Figure 3.9: Second actio: task completed.